# Programmable Mathematics for Robotic Trajectories

A verified C++ demonstrator for two-link kinematics, analytic trajectory costs, planar obstacle-clearance proxies and numerical checks with paceval.




*Marketing illustration only. The verified demonstrator is a planar two-link mathematical model, not the industrial robot shown.*

## Research result

The original whitepaper proposed prepared mathematical graphs for repeated kinematics and cost evaluations. This implementation establishes a **reproducible proof of feasibility**: 15 reusable ordinary paceval graphs and two additional TINC graphs were executed using the supplied original Linux runtime, with an independent native C++ numerical audit of every published synthetic candidate batch.

| Verified property | Observed evidence |
|---|---|
| Mathematical graph execution | 15 regular graphs; 2 extra TINC graphs |
| Synthetic waypoint experiment | 20 targets; 35 x 35 grid per target; two disk obstacles |
| Independent C++ checks | 735,000 scalar results; no configured tolerance violations |
| Independent selection oracle | 20 of 20 greedy waypoint choices reproduce the native grid optimum |
| High-precision pointwise TINC audit | 24 of 24 exact-reference values enclosed at 110 digits |

This is an **offline mathematical and planning-scoring feasibility study**, not a claim of guaranteed collision freedom, full robot planning, real-time determinism or accelerated execution. No timing comparison is made.

# 1. Scientific scope and test model

A planar robot has two revolute joints q1 and q2 with lengths l1 = 1.00 m and l2 = 0.78 m. Its end-effector location and intermediate elbow point are explicitly parameterized. This is the exact narrow model in the original research proposal.

```
x(q) = l1*cos(q1) + l2*cos(q1+q2)

y(q) = l1*sin(q1) + l2*sin(q1+q2)
```

## Extension beyond endpoint-only obstruction

The original three-page proposal supplied an endpoint-circle proxy g(q) = ||p_end - o||^2 - r^2. The new demonstrator also checks the closest point on **each of the two planar line segments** against each of two circles. For a segment with ends a and b and obstacle center o, C++ computes the clamped projection:

```
t = clamp( dot(o-a,b-a) / ||b-a||^2, 0, 1 )

g_segment = ||a + t*(b-a) - o||^2 - (r + padding)^2
```

paceval evaluates the g_segment expression as a mathematical graph using the host-provided t values. Thus these formulas evaluate precise squared-clearance proxies for the **centerline representation** and the prescribed padded disks. Actual finite-thickness robot links, 3D geometry, other obstacles, self-contact, dynamics and unknown scene objects remain outside this model.

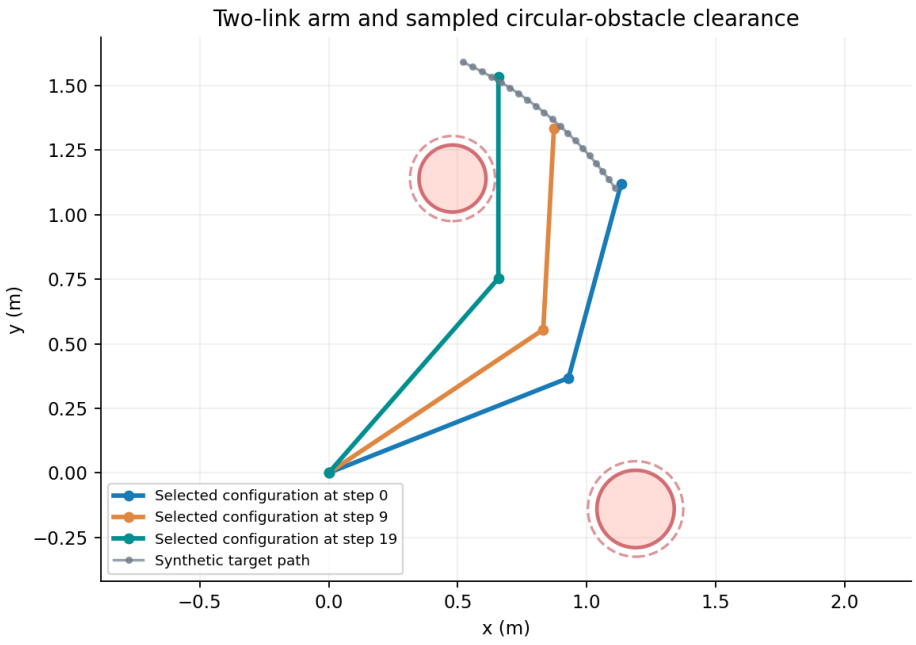



*Figure 1. Selected synthetic configurations, the idealized target path, and two padded circular obstacles.*

# 2. Mathematical graphs implemented with paceval

All 15 ordinary formulas share the ordered variable list `q1 q2 l1 l2 tx ty p1 p2 w ox oy radius t1 t2`. A graph is created once and reused for new values. For each candidate and obstacle, inputs are provided as sample-major rows; returned values from the batch API are graph-major.

| Graph(s) | Outputs computed by paceval |
|---|---|
| 1-4 | Elbow (x,y), end-effector (x,y) |
| 5-8 | 2 x 2 endpoint Jacobian entries |
| 9 | det(J) = l1*l2*sin(q2) |
| 10 | Squared endpoint tracking plus weighted squared joint-step penalty |
| 11 | Endpoint-to-padded-circle squared clearance |
| 12-13 | First and second centerline-segment squared clearance |
| 14-15 | Analytic partial derivatives of graph 10 with respect to q1 and q2 |
| TINC 1-2 | Interval-enabled copies of the endpoint x and endpoint y graphs |

```
L(q) = (x(q)-tx)^2 + (y(q)-ty)^2 + w*((q1-p1)^2+(q2-p2)^2)
```

The synthetic weighting parameter w = 0.17 scales joint-change penalties in the illustrative objective. It is not a measured robot objective or an established mechanical cost. The determinant is an indicator of planar kinematic singularity (zero when sin(q2)=0), not a substitute for general manipulability analysis.

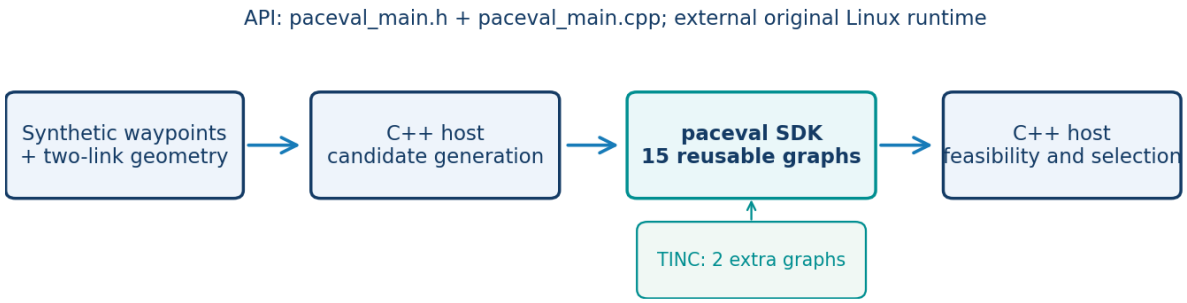



*Figure 2. Verified division of responsibilities: C++ host, direct paceval graph evaluation and optional pointwise numerical inspection.*

## API and source layout

The application directly includes `src/paceval_main.h` and compiles the original `src/paceval_main.cpp` as another translation unit. Calls to `paceval_CreateComputation`, `paceval_dGetMultipleComputationsResultsExt` and `paceval_dGetComputationResult` are followed by explicit error checks and RAII lifetime management. A compatible original Linux shared library is linked externally, not bundled.

## 3. Offline waypoint selection and trajectory output

The reference dataset contains 20 synthetic Cartesian waypoints. A fixed 35 x 35 joint-angle grid is generated at each step. For each of its 1,225 candidates, two obstacle-specific rows are submitted to paceval. Its outputs include the tracking cost, analytic derivatives and all three circle-distance expressions per obstacle.

C++ rejects candidate states with a negative padded centerline clearance, joint change beyond 0.24 rad per step, or sampled interpolated path violations. The host then selects the least-cost *admissible grid candidate for that single waypoint*. It updates the prior state before the next target. This greedy algorithm does not establish a globally optimal path over all 20 targets.

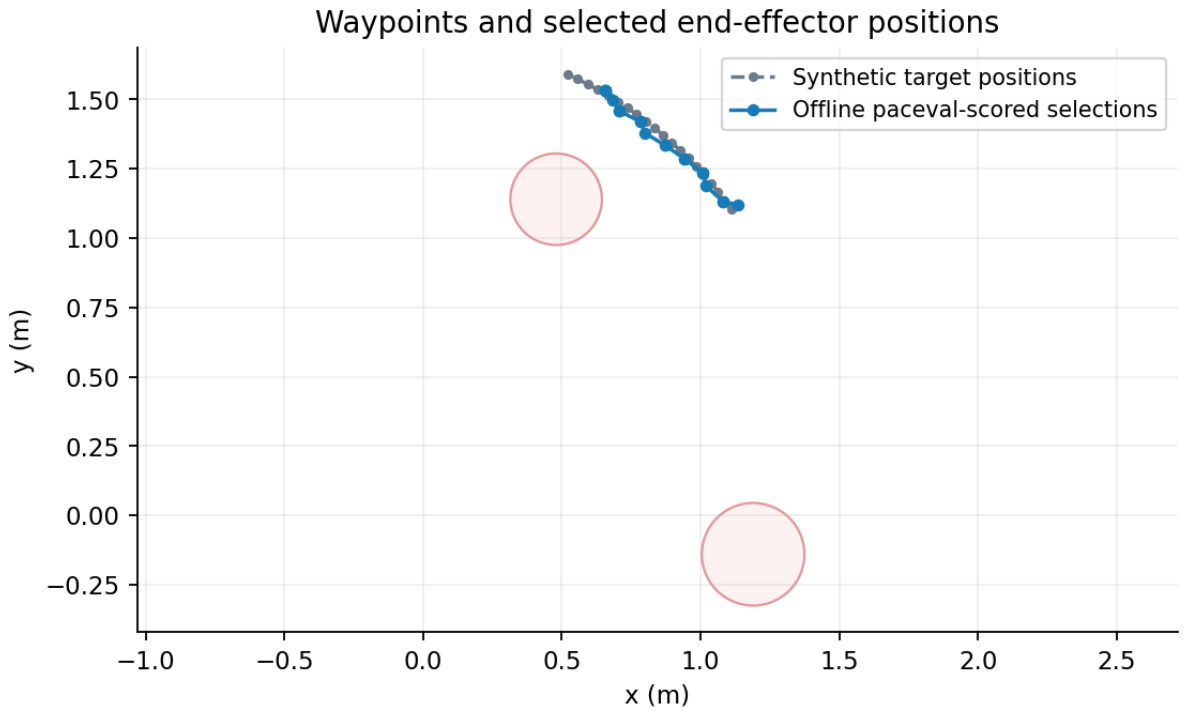



*Figure 3. Synthetic targets and accepted end-effector locations; the difference may reflect geometric constraints and finite grid resolution.*

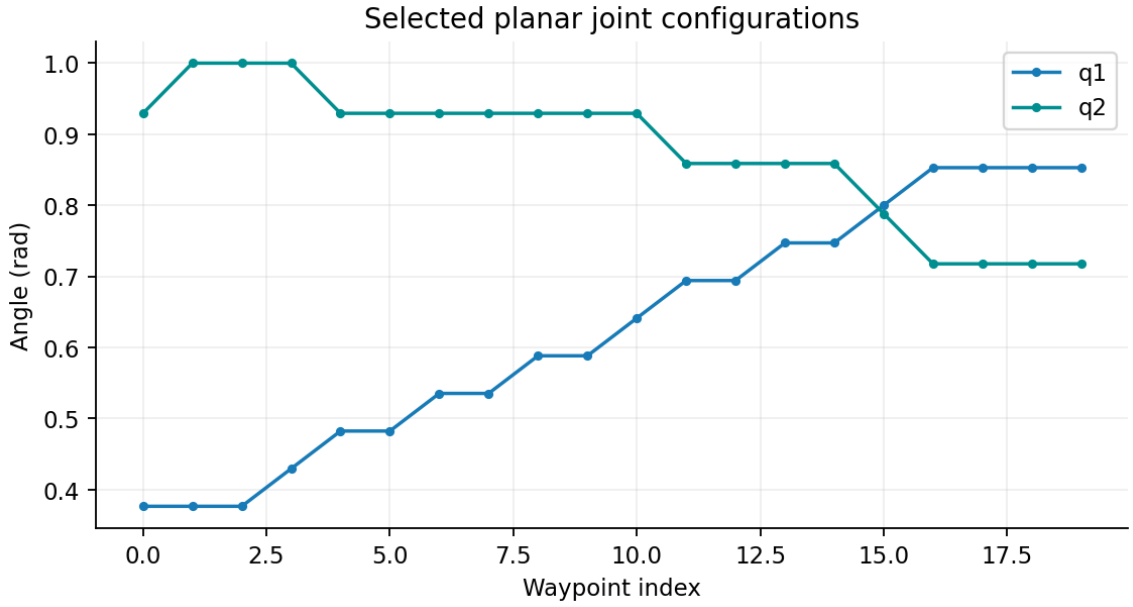



*Figure 4. Selected two-dimensional joint angles over the 20-waypoint experiment.*

# 4. Independent numerical and geometry validation

The native C++ reference independently reconstructs every kinematic, Jacobian, cost, gradient and clearance output. On the complete published run, 735,000 scalar results were compared with no configured tolerance violations. C++ derivative checks additionally use central finite differences, and projection tests cover before-segment, interior and after-segment cases.

An independent Python verifier reconstructs the exact same published synthetic two-link and disk geometry, then exhaustively checks each waypoint grid for the minimum feasible one-step cost. All 20 selected waypoints matched that independent oracle. This is a reference for the *greedy local grid-selection rule*, not a global trajectory-optimality theorem.

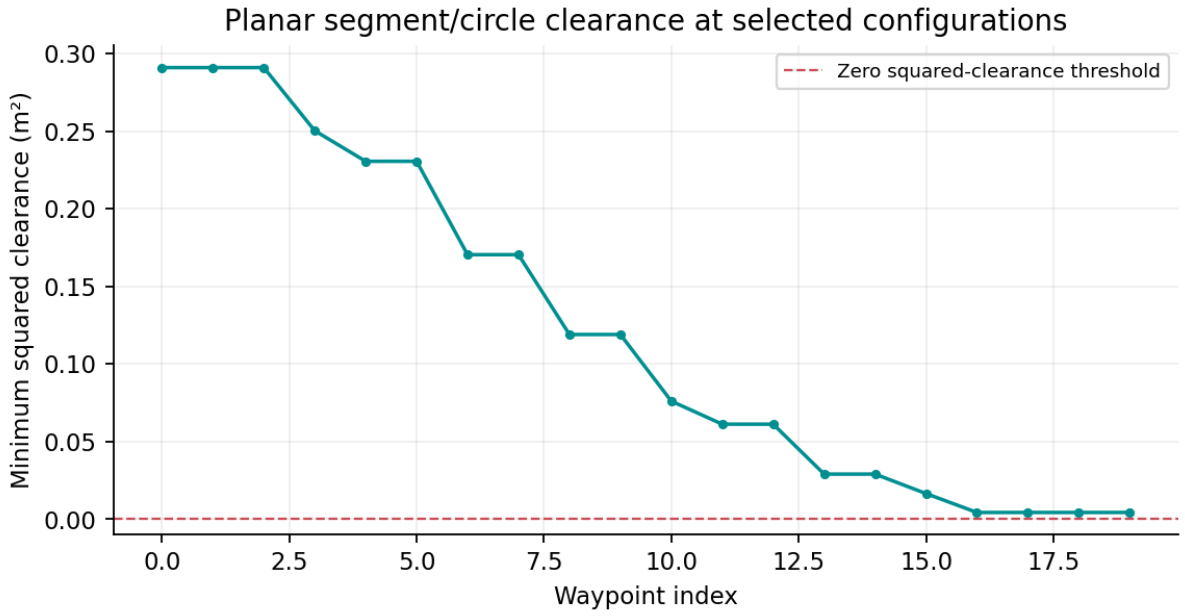



*Figure 5. Minimum squared margin among endpoint, link-one and link-two proxies for both padded circles at selected configurations.*

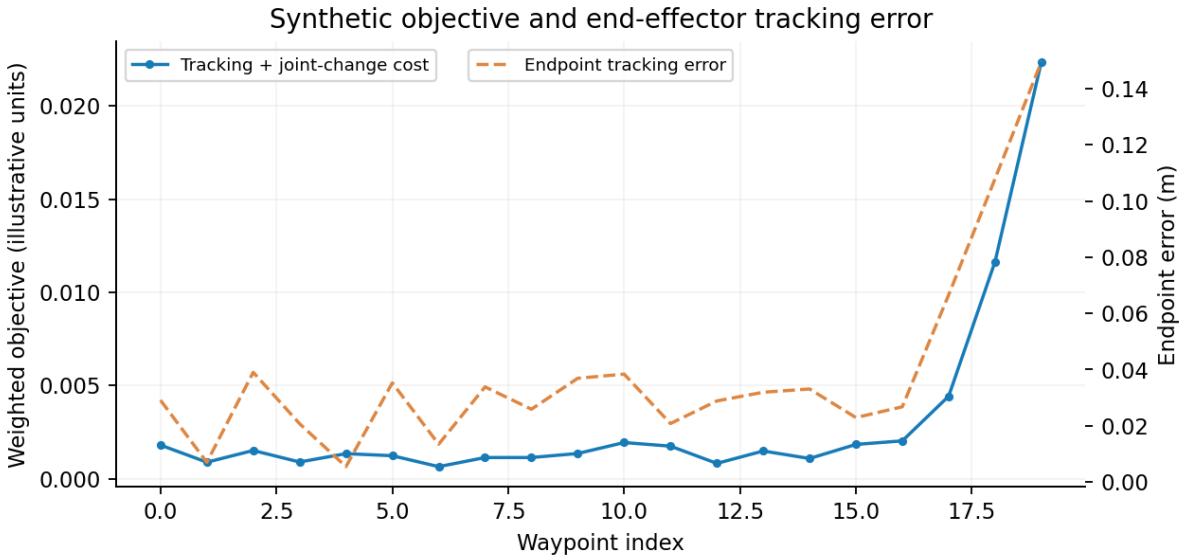



*Figure 6. Selected weighted objective values and end-effector tracking error. Both are synthetic outputs, not measured robot behavior.*

# 5. Kinematic sensitivity and pointwise TINC evaluation

The paceval graph set includes all four entries of the planar endpoint Jacobian and two symbolic derivatives of the weighted tracking cost. The native self-test differentiates x(q), y(q) and the cost numerically at representative nonsingular states.

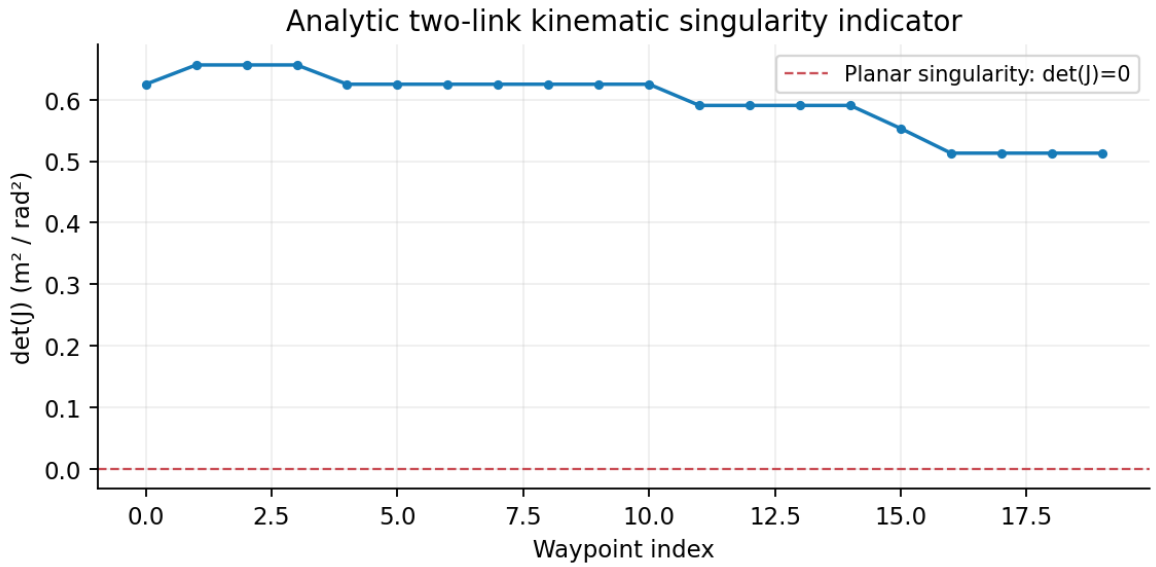



*Figure 7. Planar Jacobian determinant along the accepted synthetic configurations.*

Two additional graph handles were constructed with TINC enabled for x(q) and y(q). Their reported intervals were compared with 110-decimal-digit reference calculations, with inputs reconstructed from the same binary64 numbers presented to the C++ runtime. All 24 selected pointwise tests contained the reference value.

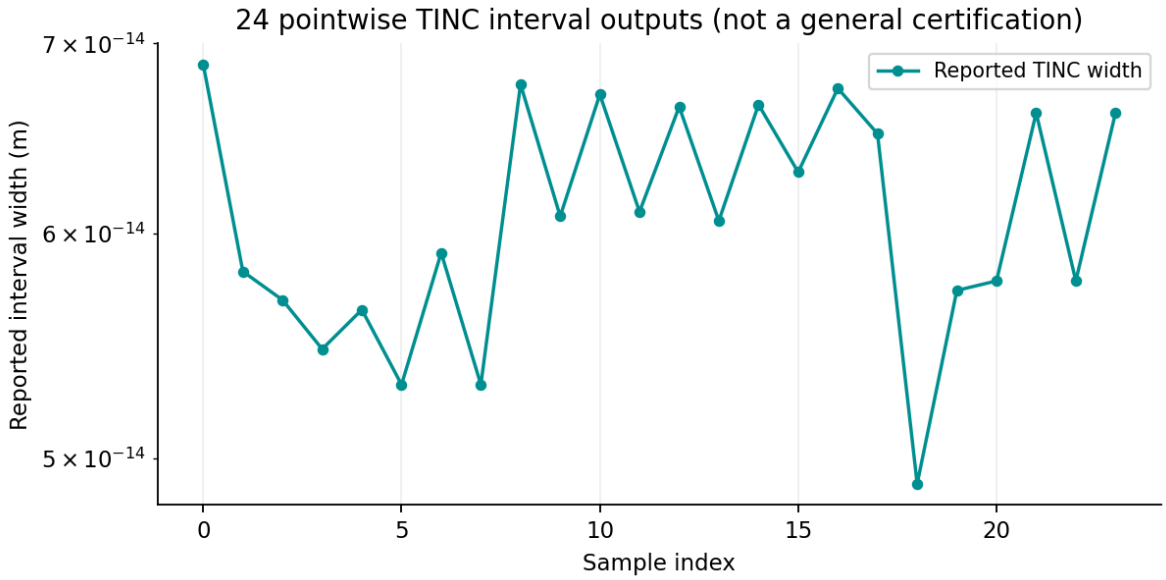



*Figure 8. Reported TINC interval widths. Pointwise inclusion is not a general proof of interval rigor.*

Statistical uncertainty, mathematical model error, encoder/sensor uncertainty and floating-point rounding are distinct concepts. None of these tests calibrates a physical sensor or certifies an arbitrary interval over a continuous input range.

# 6. Reproducibility and exact package contents

Verified environment: Linux, GCC 14.2, Clang 17, C++17 and the original paceval SDK v4.25 runtime. Both CTest checks passed under GCC, Clang and with AddressSanitizer/UndefinedBehaviorSanitizer. The complete 20-waypoint run produced byte-identical trajectory and TINC CSVs at batch sizes 256 and 37. This is a reproducibility result, not a throughput claim.

## Build from the GitHub repository root

```
export PACEVAL_LIB="/absolute/path/to/libpaceval_linux_sharedLIB.so"

cmake -S src -B build -DCMAKE_BUILD_TYPE=Release

cmake --build build -j && ctest --test-dir build --output-on-failure

./build/robotics --waypoints examples/waypoints.csv --grid 35 --batch 256
```

The default run writes `trajectory.csv` and `tinc_samples.csv`. For independent checking, pass these files to `python3 src/validate_outputs.py --trajectory trajectory.csv --tinc tinc_samples.csv --grid 35`. The pointwise high-precision audit uses Python `mpmath`; figure regeneration uses numpy and matplotlib.

| Publication file | Purpose / reproducibility link |
|---|---|
| README.md | Public research description and complete build instructions |
| src/robotics.cpp; src/CMakeLists.txt | C++ implementation and linked vendor SDK build |
| src/paceval_main.h; src/paceval_main.cpp | Original paceval SDK API layer (original notices preserved) |
| src/validate_outputs.py | Independent greedy-selection oracle and 110-digit TINC audit |
| src/make_figures.py; figures/*.png | Reproducible scientific diagrams and marketing illustration |
| examples/waypoints.csv | Deterministic 20-waypoint input |
| examples/trajectory.csv | Verified selection output for grid 35, batch 256 |
| examples/tinc_samples.csv | 24 pointwise interval outputs |

# 7. Limitations, follow-on research and conclusions

The experiment shows that **paceval can successfully implement reusable mathematics for selected trajectory-scoring and kinematic subproblems** while a clearly separated host owns the discrete planning policy. It does not replace collision-scene planning, inverse dynamics, robust optimization or robotics middleware. In this controlled setting, one can change joint candidates, targets and obstacle parameters without reconstructing the formulas.

The planar model has zero-thickness centerlines and two fixed disks, a grid-constrained greedy selector, and 13 sampled interpolation configurations per step. An input path that passes these finite checks has not been certified over continuous time. No production robot, real sensor, hardware controller, MoveIt 2, Pinocchio, FPGA or ASIC was connected or measured.

## Next stage

A robotics partner could select a documented multi-joint robot and approved collision-scene model, then place validated explicit cost/constraint callbacks into a real planning workflow. Jacobians and planner settings must remain matched across implementations. Any real-time, scalability, hardware or safety claims would require separate facility-specific verification and measurements.

## Scientific and technical references

[1] paceval UG, paceval Mathematical Engine C/C++ API v4.25: https://paceval.com/api/

[2] MoveIt 2 planning-scene source and collision interfaces: https://github.com/moveit/moveit2

[3] MoveIt 2 kinematics and collision concepts: https://github.com/moveit/moveit2_tutorials

[4] Pinocchio open-source rigid-body algorithms and derivatives: https://gepetto.github.io/doc/pinocchio/doxygen-html/

**Conclusion.** Using the supplied original Linux paceval library and direct SDK source integration, the study achieved correct mathematical graph evaluation, reproducible local candidate decisions, independent geometry/numerics checks and a successful pointwise TINC audit on the published synthetic examples. The result is a documented **feasibility demonstration**, not a robotics certification or speed claim.

Research contact: paceval UG | Munich, Germany | info@paceval.com | https://paceval.com